

ĐÁP ÁN ĐỀ THI LẬP TRÌNH MẠNG

Mã học phần IT4060 — tổng hợp từ *LTM_De-Thi.pdf*

Họ và tên: Nguyễn Huy Hoàng | MSSV: 20235336

Đáp án đề thi Lập trình mạng (IT4060)

Tổng hợp đáp án cho file *LTM_De-Thi.pdf*. Code các câu tự luận nằm trong thư mục *exams/code/*. Các chỗ có dấu (!) là đáp án cần lưu ý / khác với đáp án đã tô sẵn trong đề.

1. Ngân hàng câu hỏi (Lương Ánh Hoàng) — 95 câu

Đây là nguồn gốc của mọi mã đề trắc nghiệm. Học thuộc bảng này là đủ.

Câu	ĐA	Câu	ĐA	Câu	ĐA	Câu	ĐA	Câu	ĐA
1	d	20	c	39	d	58	d	77	d
2	b	21	d	40	c	59	a	78	a
3	b	22	a	41	d	60	a	79	a
4	b	23	a	42	d	61	b	80	b
5	d	24	d	43	d	62	a	81	c
6	d	25	c	44	a	63	b	82	a
7	a	26	a	45	d	64	d	83	a
8	d	27	c	46	b	65	d	84	b
9	a	28	d	47	c	66	a	85	a
10	c	29	d	48	b (!)	67	c	86	c
11	d	30	d	49	b	68	c	87	a
12	b	31	a	50	b	69	d	88	b
13	c	32	d	51	d	70	b	89	d
14	b	33	a	52	a	71	a	90	d
15	b	34	d (!)	53	d	72	b	91	b
16	b	35	d	54	d	73	c	92	b
17	d	36	d	55	b	74	d	93	a
18	b	37	d	56	c	75	d	94	b
19	a	38	d	57	b	76	b	95	d

Giải thích các câu quan trọng

- 1: DLL là *ws2_32.dll* → *WS2_32.DLL*.
- 2 / 3 / 30: header *Winsock2.h*; lib liên kết *WS2_32.LIB*; phân giải tên miền cần thêm *Ws2tcpip.h*.

- **13 / 14:** thêm *driver thiết bị* → tầng **Transport Protocol**; thêm *giao thức mới* → tầng **Provider**.
- **16:** giao thức hướng dòng (TCP) **không** bảo toàn biên thông điệp.
- **18:** **WSADATA** để **nhận** thông tin phiên bản WinSock có trên hệ thống (phiên bản muốn dùng truyền qua tham số **wVersionRequested**).
- **21:** **WSAGetLastError()** + **closesocket(s)** + **WSACleanup()**.
- **26:** dữ liệu >1 byte trong cấu trúc địa chỉ lưu theo **đầu to** (network byte order).
- **27 / 28:** **inet_addr("...")** đổi xâu→IP; **inet_ntoa(addr.sin_addr)** đổi IP→xâu.
- **29:** **htons** đổi cổng host→network (kiểu short).
- **34 (!):** đáp án đề là **d (cả ba)**, nhưng kỹ thuật *tường lửa KHÔNG làm bind thất bại*. Thực tế chỉ b (cổng đã bị chiếm) và c (socket không hợp lệ) gây lỗi.
- **40 / 41:** đồng bộ chặn **chỉ luồng chứa lời gọi**; bất đồng bộ **không chặn** luồng nào.
- **43:** **recv** đồng bộ **KHÔNG** chặn khi: có dữ liệu (kể cả <1024 byte), đủ 1024 byte, hoặc kết nối đã đóng → cả ba.
- **44 / 45:** 10 kết nối cần **11** socket (1 listen + 10); blocking gửi tập trung 1 luồng cho 100 kết nối cần **101** luồng.
- **46 / 47:** **connect** thành công báo qua **writelfds**; dữ liệu OOB báo qua **exceptfds**.
- **48 (!):** với 100 kết nối, do **FD_SETSIZE = 64** (Winsock) nên cần **2** luồng/2 tập **fd_set**. (Nếu thầy hỏi "select dùng mấy luồng để theo dõi nhiều socket" theo nghĩa chung thì là 1.)
- **52 / 53:** mỗi socket cần 1 **WSAEVENT** → 10 socket = **10**; **WSACreateEvent** tạo event **non-signaled, manual-reset**.
- **55:** socket client kết nối server đăng ký **FD_CONNECT | FD_WRITE | FD_READ | FD_CLOSE**.
- **58 / 59 / 60:** completion routine do **HĐH gọi trong cùng luồng** đã phát yêu cầu I/O; alertable = luồng **đang ngủ & sẵn sàng** chạy callback; đưa về alertable bằng **SleepEx** (**Sleep** thường **KHÔNG** alertable).
- **61–65:** **CSocket** **dẫn xuất** từ **CAsyncSocket**; CSocket **đồng bộ**, CAsyncSocket **bất đồng bộ**; không dùng object socket chéo luồng (64→d); xử lý sự kiện CAsyncSocket bằng cách **kế thừa + override** (OnReceive, OnConnect...).

Đoạn code điền chỗ trống (66–95)

66–72 — Nhận gói Voice/Text (TCP, struct {char type; int len; char data[65536];}) - 66 **<A>=1** (type là **char**, 1 byte) - 67 **=4** (len là **int**, 4 byte) - 68 **<C>=PACKET_TYPE_TEXT** (vì in "Text:%s") - 69 **<D>=** Phương án khác (đúng phải là **p.data+total**, không có trong lựa chọn) - 70 **<E>=p.len - total** (số byte còn lại cần nhận) - 71 **<F>=len** (cộng dồn số byte vừa nhận) - 72 **<G>=0** (kết thúc chuỗi **\0**)

73–79 — Nhận UDP + kiểm checksum (XOR) - 73 **<A>=sizeof(from)** - 74 **=total < p.len** - 75 **<C>=** Cả ba đều sai (đúng phải là **p.data+total**) - 76 **<D>=p.len - total** - 77 **<E>=(sockaddr*)&from** - 78 **<F>=^** (toán tử XOR) - 79 **<G>=i*2** (mỗi **unsigned short** = 2 byte)

80–86 — Server quản lý kết nối bằng danh sách liên kết kép - 80 **<A>=c** (lưu kết quả **accept**) - 81 **=&cAddrLen** - 82 **<C>=new Connection** - 83 **<D>=pTmp** (**pCur->pNext = pTmp**) - 84 **<E>=pCur** (**pTmp->pPrev = pCur**) - 85 **<F>=c** (**pTmp->s = c**) - 86 **<G>=pTmp** (tham số cho **ReceiverThread**)

87–90 — Phân giải tên miền bằng `getaddrinfo` - 87 `<A>="8888"` (service là **xâu**) - 88 `<B>=&info` - 89 `<C>=pCur->ai_addr` - 90 `<D>=inet_ntoa(addr.sin_addr)`

91–95 — HTTP server tách URL (đọc tới `\r\n\r\n`) - 91 `<A>=command+len` - 92 `<B>=len` - 93 `<C>===` (if (pos == 0) → không tìm thấy → Invalid) - 94 `<D>=command+4` (bỏ "GET ") - 95 `<E>=pos-command-4`

2. Mã đề trắc nghiệm

Đề 515 (40 câu)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
d	b	b	d	b	a	d	a	c	d	b	c	b	b	b	d	b	a	c	d

21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
a	a	d	c	a	c	d	d	d	d	a	d	d	a	b	d	a	c	a	b

- Q5: bắt gói tin → **b. Network Monitor** (Cain/Netcat không phải công cụ bắt gói).
- Q20 & Q23: đáp án đúng (WSAGetLastError+closesocket+WSACleanup / socket UDP đúng) **không** có trong lựa chọn → chọn **d (Không phương án nào đúng)**.
- Q31–35 đảo chuỗi: `<A>=len`, `<B>=str[i]`, `<C>=len-i-1`, `<D>=c`, `<E>=s`.
- Q36–40 đọc & gửi file: `<A>=!feof(fp)`, `<B>=buff`, `<C>=fp`, `<D>=s`, `<E>=len`.

Đề 217 (40 câu)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
d	d	b	b	d	b	b	b	a	a	c	d	c	b	b	b	d	b	d	a

21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
c	c	a	d	d	c	a	c	a	d	d	a	c	a	b	a	d	d	a	b

- Q8: bắt gói tin → **b. Wireshark**.
- Q24 (socket UDP) & Q25 (connect code): đáp án đúng không có → **d**.
- Q31–35 đọc & gửi file: `!feof(fp)`, `buff`, `fp`, `s`, `len`.
- Q36–40 đảo chuỗi: `len`, `str[i]`, `len-i-1`, `c`, `s`.

Đề 247 — Giữa kỳ (30 trắc nghiệm)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
d	b	b	d	c	b	a	b	b	d	a	c	d	b	b

16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
b	b	d	a	a	c	a	c	c	a	d	d	d	d	d

- **Phần 2 (tự luận):** viết `FileClient` mô hình **blocking** → `exams/code/FileClient_blocking.cpp`.

Đề 132 — Giữa kỳ (30 trắc nghiệm)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
d	b	c	b	b	b	b	d	a	b	c	d	d	b	b

16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
a	b	d	a	a	d	c	a	c	c	a	d	d	d	d

- **Phần 2 (tự luận):** viết `FileClient` mô hình **select** → `exams/code/FileClient_select.cpp`.

3. Câu hỏi online (trắc nghiệm trên hệ thống)

Câu hỏi	Đáp án đúng
<code>fork(); fork();</code> tạo bao nhiêu tiến trình con?	3 ($2^2 - 1$)
Giao thức mail client (Outlook) gửi email	SMTP
pthread: <code>pthread_create</code> → <code>pthread_join</code> → in	Child thread. rồi Main thread. (!)
Giá trị socket lớn nhất <code>select</code> chờ được (Linux)	1023
HTTP upload file lớn lên server	POST
<code>pollfd</code> — trường đăng ký mặt nạ sự kiện	events (!)
Khi nào cần xử lý xung đột luồng (đa chọn)	đọc/ghi biến toàn cục + ghi file
Trình duyệt tải nội dung trang web	HTTP
Nhận định sai về đa tiến trình	"Các tiến trình dùng chung biến toàn cục "
Outlook nhận email từ mail server (đa chọn)	POP3 + IMAP4
Phục vụ client song song (đa chọn)	<code>fork()</code> + luồng
<code>if(fork()) n=n+1; else n=n+2; (n=1)</code>	n = 2 và n = 3
pthread truyền <code>&n</code> , hàm sửa biến cục bộ	10 (không đổi biến gốc)

(!) Hai câu đáp án tô sẵn trong PDF bị SAI: 1. **pthread + join**: Vì `pthread_join` chờ luồng con xong rồi mới in, kết quả đúng là `Child thread.` → `Main thread.` (PDF tô "Main thread. / Child thread." là sai). 2. **pollfd**: trường đăng ký sự kiện cần thăm dò là `.events` (kernel trả về ở `.revents`). PDF tô "cả events và revents không dùng" là sai.

4. Câu tự luận (code)

Đề	Yêu cầu	File
Giữa kỳ 247	FileClient — blocking	<code>code/FileClient_blocking.cpp</code>
Giữa kỳ 132	FileClient — select	<code>code/FileClient_select.cpp</code>
Cuối kỳ 20122-002	Server xác thực USER/PASS	<code>code/AuthServer.cpp</code>
Đề 1 (chat)	ChatClient/Server — blocking	<code>code/ChatClient_blocking.cpp</code> , <code>code/ChatServer_blocking.cpp</code>
Đề 2–5 (chat)	select / WSAEventSelect / Overlapped Event / Completion Routine	xem ghi chú trong <code>code/ChatServer_blocking.cpp</code>

Phu luc — Ma nguon cac cau tu luan

FileClient_blocking.cpp — FileClient — mô hình blocking (de 247)

```

// FileClient - mô hình BLOCKING (de giữa kỳ 247)
// Biên dịch trên Windows (Visual Studio hoặc MinGW):
// cl FileClient_blocking.cpp ws2_32.lib
// g++ FileClient_blocking.cpp -lws2_32 -o FileClient
//
// Giao thức:
// Client gửi: "GET <TênFile>\n"
// Server trả: "FAILED\n<ThôngBáoLỗi>\n\n"
//             hoặc "OK\n<FileSize>\n<NộiDungFile>\n\n"
// Yêu cầu: tại được nhiều file cùng lúc, từ nhiều server -> mỗi file 1 luồng.

#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#pragma comment(lib, "ws2_32.lib")

#define SERVER_PORT "8888"

// Tham số truyền cho mỗi luồng tại file
struct DownloadInfo {
    char host[256];
    char fileName[256];
};

// Đọc dung n byte từ socket (blocking). Trả về số byte đọc được, <=0 nếu lỗi/dừng.
int recvAll(SOCKET s, char *buf, int n) {
    int total = 0;
    while (total < n) {
        int ret = recv(s, buf + total, n - total, 0);
        if (ret <= 0) return ret; // lỗi hoặc kết nối dừng
        total += ret;
    }
    return total;
}

// Đọc từng byte cho đến khi gặp ký tự '\n'. Không chép '\n' vào line.
// Trả về 1 nếu thành công, <=0 nếu lỗi/dừng kết nối.
int recvLine(SOCKET s, char *line, int maxLen) {
    int i = 0;
    char c;
    while (i < maxLen - 1) {
        int ret = recv(s, &c, 1, 0);
        if (ret <= 0) return ret;
        if (c == '\n') break;
        line[i++] = c;
    }
    line[i] = 0;
    return 1;
}

DWORD WINAPI DownloadThread(LPVOID param) {
    DownloadInfo *info = (DownloadInfo *)param;

    // 1. Phân giải tên miền / địa chỉ
    addrinfo hints, *res = NULL;
    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_protocol = IPPROTO_TCP;

    if (getaddrinfo(info->host, SERVER_PORT, &hints, &res) != 0) {
        printf("[%s] Không phân giải được địa chỉ server\n", info->fileName);
        free(info);
        return 1;
    }
}

```

```

// 2. Tao socket va ket noi
SOCKET s = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
if (s == INVALID_SOCKET) {
    printf("[%s] Loi tao socket: %d\n", info->fileName, WSAGetLastError());
    freeaddrinfo(res);
    free(info);
    return 1;
}
if (connect(s, res->ai_addr, (int)res->ai_addrlen) == SOCKET_ERROR) {
    printf("[%s] Loi ket noi: %d\n", info->fileName, WSAGetLastError());
    closesocket(s);
    freeaddrinfo(res);
    free(info);
    return 1;
}
freeaddrinfo(res);

// 3. Gui yeu cau "GET <TenFile>\n"
char request[512];
int reqLen = sprintf(request, "GET %s\n", info->fileName);
if (send(s, request, reqLen, 0) == SOCKET_ERROR) {
    printf("[%s] Loi gui yeu cau: %d\n", info->fileName, WSAGetLastError());
    closesocket(s);
    free(info);
    return 1;
}

// 4. Doc dong trang thai dau tien ("OK" hoac "FAILED")
char status[64];
if (recvLine(s, status, sizeof(status)) <= 0) {
    printf("[%s] Server dong ket noi\n", info->fileName);
    closesocket(s);
    free(info);
    return 1;
}

if (strcmp(status, "FAILED") == 0) {
    char errMsg[1024];
    recvLine(s, errMsg, sizeof(errMsg)); // dong thong bao loi
    printf("[%s] Server bao loi: %s\n", info->fileName, errMsg);
    closesocket(s);
    free(info);
    return 0;
}

if (strcmp(status, "OK") != 0) {
    printf("[%s] Phan hoi khong hop le: %s\n", info->fileName, status);
    closesocket(s);
    free(info);
    return 1;
}

// 5. Doc kich thuc file
char sizeLine[64];
if (recvLine(s, sizeLine, sizeof(sizeLine)) <= 0) {
    closesocket(s);
    free(info);
    return 1;
}
long fileSize = atol(sizeLine);

// 6. Doc dung fileSize byte noi dung va ghi ra dia
FILE *fp = fopen(info->fileName, "wb");
if (!fp) {
    printf("[%s] Khong mo duoc file de ghi\n", info->fileName);
    closesocket(s);
    free(info);
    return 1;
}

char buf[4096];
long remain = fileSize;

```

```

while (remain > 0) {
    int want = (remain < (long)sizeof(buf)) ? (int)remain : (int)sizeof(buf);
    int ret = recv(s, buf, want, 0);
    if (ret <= 0) break;
    fwrite(buf, 1, ret, fp);
    remain -= ret;
}
fclose(fp);

if (remain == 0)
    printf("[%s] Tai xong (%ld byte)\n", info->fileName, fileSize);
else
    printf("[%s] Tai loi, thieu %ld byte\n", info->fileName, remain);

closesocket(s);
free(info);
return 0;
}

int main() {
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("WSAStartup loi\n");
        return 1;
    }

    printf("FileClient (blocking). Nhap rong o ten file de thoat.\n");
    while (1) {
        DownloadInfo *info = (DownloadInfo *)malloc(sizeof(DownloadInfo));

        printf("\nDia chi server: ");
        if (!fgets(info->host, sizeof(info->host), stdin)) { free(info); break; }
        info->host[strcspn(info->host, "\r\n")] = 0;

        printf("Ten file: ");
        if (!fgets(info->fileName, sizeof(info->fileName), stdin)) { free(info); break; }
        info->fileName[strcspn(info->fileName, "\r\n")] = 0;

        if (strlen(info->fileName) == 0) { free(info); break; }

        // Moi yeu cau tai mot luong rieng -> tai nhieu file cung luc
        HANDLE h = CreateThread(NULL, 0, DownloadThread, info, 0, NULL);
        if (h) CloseHandle(h);
        else { free(info); }
    }

    Sleep(2000); // cho cac luong tai con lai hoan tat (don gian hoa)
    WSACleanup();
    return 0;
}

```

FileClient_select.cpp — FileClient — mô hình select (đề 132)

```
// FileClient - mô hình SELECT (đề giữa kỳ 132)
// Biên dịch trên Windows:
//   cl FileClient_select.cpp ws2_32.lib
//   g++ FileClient_select.cpp -lws2_32 -o FileClient
//
// Giao thức: giống đề 247
// Client gửi: "GET <TênFile>\n"
// Server trả: "FAILED\n<ThôngBaoLoi>\n\n"
//             hoặc "OK\n<FileSize>\n<NộiDungFile>\n\n"
// Mô hình select: 1 luồng duy nhất, socket bất động chờ, dùng select() để
// thực hiện tại NHIEU file cùng lúc từ nhiều server.
//
// Đơn giản hóa: nhập trước danh sách (server, tên file), dòng trống để bắt đầu tại.

#include <winsock2.h>
#include <ws2tcpip.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <string>
#include <vector>

#pragma comment(lib, "ws2_32.lib")

#define SERVER_PORT "8888"

enum State { ST_CONNECTING, ST_RECEIVING, ST_DONE };
enum Phase { PH_STATUS, PH_SIZE, PH_CONTENT, PH_FAILMSG };

struct Conn {
    SOCKET s;
    State state;
    Phase phase;
    std::string host;
    std::string fileName;
    std::string buf;      // bỏ đệm tích lũy ở pha header
    long fileSize;
    long received;
    FILE *fp;
};

void closeConn(Conn *c) {
    if (c->fp) { fclose(c->fp); c->fp = NULL; }
    if (c->s != INVALID_SOCKET) { closesocket(c->s); c->s = INVALID_SOCKET; }
    c->state = ST_DONE;
}

// Phân tích dữ liệu đã nhận được trong c->buf (gọi sau mỗi lần recv)
void process(Conn *c) {
    while (true) {
        if (c->phase == PH_STATUS || c->phase == PH_SIZE || c->phase == PH_FAILMSG) {
            size_t pos = c->buf.find('\n');
            if (pos == std::string::npos) return;      // chưa đủ 1 dòng, chờ thêm
            std::string line = c->buf.substr(0, pos);
            c->buf.erase(0, pos + 1);

            if (c->phase == PH_STATUS) {
                if (line == "OK") c->phase = PH_SIZE;
                else if (line == "FAILED") c->phase = PH_FAILMSG;
                else { printf("[%s] Phản hồi sai: %s\n", c->fileName.c_str(), line.c_str());
                    closeConn(c); return; }
            } else if (c->phase == PH_SIZE) {
                c->fileSize = atol(line.c_str());
                c->received = 0;
                c->fp = fopen(c->fileName.c_str(), "wb");
                if (!c->fp) { printf("[%s] Không mở được file ghi\n", c->fileName.c_str());
                    closeConn(c); return; }
                c->phase = PH_CONTENT;
            } else { // PH_FAILMSG

```

```

        printf("[%s] Server bao loi: %s\n", c->fileName.c_str(), line.c_str());
        closeConn(c);
        return;
    }
} else { // PH_CONTENT
    long remain = c->fileSize - c->received;
    if (remain <= 0) {
        printf("[%s] Tai xong (%ld byte)\n", c->fileName.c_str(), c->fileSize);
        closeConn(c);
        return;
    }
    long avail = (long)c->buf.size();
    long take = (avail < remain) ? avail : remain;
    if (take > 0) {
        fwrite(c->buf.data(), 1, take, c->fp);
        c->received += take;
        c->buf.erase(0, take);
    }
    if (c->received >= c->fileSize) {
        printf("[%s] Tai xong (%ld byte)\n", c->fileName.c_str(), c->fileSize);
        closeConn(c);
        return;
    }
    if (c->buf.empty()) return; // het du lieu trong bo dem, cho recv tiep
}
}

int main() {
    WSADATA wsa;
    WSASStartup(MAKEWORD(2, 2), &wsa);

    std::vector<Conn *> conns;

    printf("FileClient (select). Nhap danh sach can tai, dong rong de bat dau.\n");
    while (true) {
        char host[256], file[256];
        printf("Dia chi server (rong = xong): ");
        if (!fgets(host, sizeof(host), stdin)) break;
        host[strcspn(host, "\r\n")] = 0;
        if (strlen(host) == 0) break;
        printf("Ten file: ");
        if (!fgets(file, sizeof(file), stdin)) break;
        file[strcspn(file, "\r\n")] = 0;
        if (strlen(file) == 0) break;

        // Phan giai + tao socket bat dong bo + connect
        addrinfo hints, *res = NULL;
        memset(&hints, 0, sizeof(hints));
        hints.ai_family = AF_INET;
        hints.ai_socktype = SOCK_STREAM;
        hints.ai_protocol = IPPROTO_TCP;
        if (getaddrinfo(host, SERVER_PORT, &hints, &res) != 0) {
            printf("[%s] Khong phan giai duoc %s\n", file, host);
            continue;
        }
        SOCKET s = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
        u_long mode = 1;
        ioctlsocket(s, FIONBIO, &mode); // chuyen sang bat dong bo
        connect(s, res->ai_addr, (int)res->ai_addrlen); // se tra WSAEWOULDBLOCK
        freeaddrinfo(res);

        Conn *c = new Conn();
        c->s = s; c->state = ST_CONNECTING; c->phase = PH_STATUS;
        c->host = host; c->fileName = file;
        c->fileSize = 0; c->received = 0; c->fp = NULL;
        conns.push_back(c);
    }

    // Vong lap select: chay den khi tat ca ket noi xong
    while (true) {
        fd_set readfds, writefds, exceptfds;
        FD_ZERO(&readfds); FD_ZERO(&writefds); FD_ZERO(&exceptfds);
    }
}

```

```

int active = 0;
for (size_t i = 0; i < conns.size(); i++) {
    Conn *c = conns[i];
    if (c->state == ST_CONNECTING) {
        FD_SET(c->s, &writefds);
        FD_SET(c->s, &exceptfds); // bat su kien connect that bai
        active++;
    } else if (c->state == ST_RECEIVING) {
        FD_SET(c->s, &readfds);
        active++;
    }
}
if (active == 0) break; // xong het

if (select(0, &readfds, &writefds, &exceptfds, NULL) == SOCKET_ERROR) {
    printf("select loi: %d\n", WSAGetLastError());
    break;
}

for (size_t i = 0; i < conns.size(); i++) {
    Conn *c = conns[i];
    if (c->state == ST_DONE) continue;

    // Connect that bai
    if (c->state == ST_CONNECTING && FD_ISSET(c->s, &exceptfds)) {
        printf("[%s] Ket noi that bai\n", c->fileName.c_str());
        closeConn(c);
        continue;
    }
    // Connect thanh cong -> gui yeu cau, chuyen sang nhan
    if (c->state == ST_CONNECTING && FD_ISSET(c->s, &writefds)) {
        std::string req = "GET " + c->fileName + "\n";
        send(c->s, req.c_str(), (int)req.size(), 0);
        c->state = ST_RECEIVING;
        continue;
    }
    // Co du lieu de doc
    if (c->state == ST_RECEIVING && FD_ISSET(c->s, &readfds)) {
        char tmp[4096];
        int ret = recv(c->s, tmp, sizeof(tmp), 0);
        if (ret > 0) {
            c->buf.append(tmp, ret);
            process(c);
        } else if (ret == 0) {
            // server dong
            if (c->phase == PH_CONTENT && c->received < c->fileSize)
                printf("[%s] Server dong som, thieu %ld byte\n",
                    c->fileName.c_str(), c->fileSize - c->received);
            closeConn(c);
        } else {
            if (WSAGetLastError() != WSAEWOULDBLOCK) closeConn(c);
        }
    }
}
}

for (size_t i = 0; i < conns.size(); i++) delete conns[i];
WSACleanup();
return 0;
}

```

AuthServer.cpp — AuthServer — USER/PASS (de cuoi ky)

```

// AuthServer - de cuoi ky IT4060-20122-CuoiKy-002
// Bien dich tren Windows:
// cl AuthServer.cpp ws2_32.lib
// g++ AuthServer.cpp -lws2_32 -o AuthServer
//
// Nhiem vu:
// - Tao socket, lang nghe, chap nhan ket noi tu client (moi client 1 luong).
// - Nhan lenh tu client va xu ly:
//     USER:<ten>  -> co user khong? "No user found" / "OK"
//     PASS:<mk>   -> chua gui USER: "Please send USER first"
//                  mat khau dung: "OK"
//                  mat khau sai: "Invalid Password"
// - File data.txt o server, moi dong: <username> <password>

#include <winsock2.h>
#include <windows.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#pragma comment(lib, "ws2_32.lib")

#define SERVER_PORT 8888
#define DATA_FILE  "data.txt"

// Tim user trong data.txt. Neu thay, chep mat khau vao outPass va tra ve 1.
int findUser(const char *user, char *outPass, int outPassSize) {
    FILE *fp = fopen(DATA_FILE, "rt");
    if (!fp) return 0;
    char line[512], u[256], p[256];
    int found = 0;
    while (fgets(line, sizeof(line), fp)) {
        line[strcspn(line, "\r\n")] = 0;
        if (sscanf(line, "%255s %255s", u, p) == 2) {
            if (strcmp(u, user) == 0) {
                strncpy(outPass, p, outPassSize - 1);
                outPass[outPassSize - 1] = 0;
                found = 1;
                break;
            }
        }
    }
    fclose(fp);
    return found;
}

// Doc 1 dong (ket thuc bang '\n') tu socket. Tra ve 1 neu OK, <=0 neu dong/loi.
int recvLine(SOCKET s, char *line, int maxLen) {
    int i = 0;
    char c;
    while (i < maxLen - 1) {
        int ret = recv(s, &c, 1, 0);
        if (ret <= 0) return ret;
        if (c == '\n') break;
        if (c != '\r') line[i++] = c;
    }
    line[i] = 0;
    return 1;
}

void sendMsg(SOCKET s, const char *msg) {
    send(s, msg, (int)strlen(msg), 0);
}

DWORD WINAPI ClientThread(LPVOID param) {
    SOCKET client = (SOCKET)(UINT_PTR)param;

    int hasUser = 0;           // da gui USER hop le chua
    char curUser[256] = "";    // user dang chon

```

```

char curPass[256] = ""; // mat khau cua user do (trong data.txt)

char line[512];
while (recvLine(client, line, sizeof(line)) > 0) {
    if (strncmp(line, "USER:", 5) == 0) {
        const char *name = line + 5;
        char pass[256];
        if (findUser(name, pass, sizeof(pass))) {
            strncpy(curUser, name, sizeof(curUser) - 1);
            strncpy(curPass, pass, sizeof(curPass) - 1);
            hasUser = 1;
            sendMsg(client, "OK\n");
        } else {
            sendMsg(client, "No user found\n");
        }
    } else if (strncmp(line, "PASS:", 5) == 0) {
        const char *pass = line + 5;
        if (!hasUser) {
            sendMsg(client, "Please send USER first\n");
        } else if (strcmp(pass, curPass) == 0) {
            sendMsg(client, "OK\n");
        } else {
            sendMsg(client, "Invalid Password\n");
        }
    } else {
        sendMsg(client, "Unknown command\n");
    }
}

closesocket(client);
return 0;
}

int main() {
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("WSAStartup loi\n");
        return 1;
    }

    SOCKET listenSock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (listenSock == INVALID_SOCKET) {
        printf("Loi tao socket: %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    sockaddr_in serverAddr;
    memset(&serverAddr, 0, sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(SERVER_PORT);

    if (bind(listenSock, (sockaddr *)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
        printf("Loi bind: %d\n", WSAGetLastError());
        closesocket(listenSock);
        WSACleanup();
        return 1;
    }

    listen(listenSock, SOMAXCONN);
    printf("AuthServer dang lang nghe o cong %d ...\n", SERVER_PORT);

    while (1) {
        sockaddr_in clientAddr;
        int len = sizeof(clientAddr);
        SOCKET client = accept(listenSock, (sockaddr *)&clientAddr, &len);
        if (client == INVALID_SOCKET) continue;

        printf("Client ket noi: %s:%d\n",
            inet_ntoa(clientAddr.sin_addr), ntohs(clientAddr.sin_port));

        HANDLE h = CreateThread(NULL, 0, ClientThread, (LPVOID)(UINT_PTR)client, 0, NULL);
        if (h) CloseHandle(h);
    }
}

```

```
        else closesocket(client);  
    }  
  
    closesocket(listenSock);  
    WSACleanup();  
    return 0;  
}
```

data.txt — data.txt (du lieu mau cho AuthServer)

```
admin 123456
alice wonderland
bob secret
hoang nhh4075
```

ChatServer_blocking.cpp — ChatServer — blocking (de 1)

```

// ChatServer - mo hinh BLOCKING (de chat - DE 1, Cau 2)
// Bien dich tren Windows:
// cl ChatServer_blocking.cpp ws2_32.lib
// g++ ChatServer_blocking.cpp -lws2_32 -o ChatServer
//
// Giao thuc (cong TCP 8888):
// struct Packet { int iType; int iLength; char szNickname[32]; char bData[1024]; }
// iType = 0: dang nhap (client->server), szNickname = nickname dang ky.
// iType = 1: chat. client->server: szNickname = nickname DICH, bData = cau thoai.
//           Server sua szNickname thanh nickname NGUOI GUI roi chuyen tiep den dich.
// iType = 2: list. client->server: xin danh sach. server->client: bData chua
//           danh sach nickname, moi nickname chiem dung 32 byte (dem NULL), iLength = n*32.
// Server chu dong gui lai list (type 2) cho TAT CA client moi khi co client
// dang nhap / dang xuat.
//
// =====
// GHI CHU cac mo hinh khac (DE 2-5) - cung giao thuc, chi khac cach vao/ra:
// - DE 2 (select):      1 luong, dat cac SOCKET vao fd_set, goi select() de
//                       biet socket nao co du lieu / co ket noi moi (readfds).
// - DE 3 (WSAEventSelect): moi SOCKET gan 1 WSAEVENT bang WSAEventSelect(s,ev,
//                       FD_ACCEPT|FD_READ|FD_CLOSE), cho bang WSAWaitForMultipleEvents,
//                       WSAEnumNetworkEvents de biet su kien.
// - DE 4 (Overlapped Event): moi WSAREcv/WSASend kem 1 WSAOVERLAPPED co hEvent,
//                       cho bang WSAWaitForMultipleEvents, WSAGetOverlappedResult.
// - DE 5 (Completion Routine): WSAREcv/WSASend kem completion routine (callback),
//                       luong phai o trang thai alertable (SleepEx/WSAWaitForMultipleEventsEx).
// Toan bo logic xu ly goi tin (dang nhap, chuyen tiep chat, broadcast list) giu nguyen.
// =====

#include <winsock2.h>
#include <windows.h>
#include <stdio.h>
#include <string.h>

#pragma comment(lib, "ws2_32.lib")

#define SERVER_PORT 8888
#define MAX_CLIENTS 64
#define NICK_LEN 32
#define DATA_LEN 1024

struct Packet {
    int iType;
    int iLength;
    char szNickname[NICK_LEN];
    char bData[DATA_LEN];
};

struct ClientInfo {
    SOCKET s;
    char nickname[NICK_LEN];
    int used;
};

ClientInfo g_clients[MAX_CLIENTS];
CRITICAL_SECTION g_cs;

// --- Gui/nhan tron ven 1 goi tin (blocking) ---
int sendPacket(SOCKET s, Packet *p) {
    char *buf = (char *)p;
    int total = 0, n = sizeof(Packet);
    while (total < n) {
        int ret = send(s, buf + total, n - total, 0);
        if (ret <= 0) return ret;
        total += ret;
    }
    return total;
}

```

```

int recvPacket(SOCKET s, Packet *p) {
    char *buf = (char *)p;
    int total = 0, n = sizeof(Packet);
    while (total < n) {
        int ret = recv(s, buf + total, n - total, 0);
        if (ret <= 0) return ret;
        total += ret;
    }
    return total;
}

// --- Quan ly danh sach client ---
void addClient(SOCKET s, const char *nick) {
    EnterCriticalSection(&g_cs);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (!g_clients[i].used) {
            g_clients[i].used = 1;
            g_clients[i].s = s;
            strncpy(g_clients[i].nickname, nick, NICK_LEN - 1);
            g_clients[i].nickname[NICK_LEN - 1] = 0;
            break;
        }
    }
    LeaveCriticalSection(&g_cs);
}

void removeClient(SOCKET s) {
    EnterCriticalSection(&g_cs);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (g_clients[i].used && g_clients[i].s == s) {
            g_clients[i].used = 0;
            break;
        }
    }
    LeaveCriticalSection(&g_cs);
}

// Tim nickname nguoi gui theo socket
void getNickBySock(SOCKET s, char *out) {
    EnterCriticalSection(&g_cs);
    out[0] = 0;
    for (int i = 0; i < MAX_CLIENTS; i++)
        if (g_clients[i].used && g_clients[i].s == s) {
            strncpy(out, g_clients[i].nickname, NICK_LEN);
            break;
        }
    LeaveCriticalSection(&g_cs);
}

// Tim socket dich theo nickname
SOCKET getSockByNick(const char *nick) {
    SOCKET found = INVALID_SOCKET;
    EnterCriticalSection(&g_cs);
    for (int i = 0; i < MAX_CLIENTS; i++)
        if (g_clients[i].used && strcmp(g_clients[i].nickname, nick, NICK_LEN) == 0) {
            found = g_clients[i].s;
            break;
        }
    LeaveCriticalSection(&g_cs);
    return found;
}

// Tao goi tin danh sach (type 2): moi nickname 32 byte, dem NULL
void buildListPacket(Packet *p) {
    memset(p, 0, sizeof(Packet));
    p->iType = 2;
    int count = 0;
    EnterCriticalSection(&g_cs);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (g_clients[i].used) {
            // bData[count*32] chua nickname thu (count+1)
            strncpy(p->bData + count * NICK_LEN, g_clients[i].nickname, NICK_LEN);
            count++;
        }
    }
}

```

```

    }
}
LeaveCriticalSection(&g_cs);
p->iLength = count * NICK_LEN;
}

// Gui danh sach hien tai den TAT CA client
void broadcastList() {
    Packet p;
    buildListPacket(&p);

    // Chup nhanh danh sach socket de tranh giu khoa khi gui (blocking)
    SOCKET socks[MAX_CLIENTS];
    int n = 0;
    EnterCriticalSection(&g_cs);
    for (int i = 0; i < MAX_CLIENTS; i++)
        if (g_clients[i].used) socks[n++] = g_clients[i].s;
    LeaveCriticalSection(&g_cs);

    for (int i = 0; i < n; i++) sendPacket(socks[i], &p);
}

DWORD WINAPI ClientThread(LPVOID param) {
    SOCKET client = (SOCKET)(UINT_PTR)param;
    Packet p;

    // 1. Goi dau tien phai la dang nhap (type 0)
    if (recvPacket(client, &p) <= 0 || p.iType != 0) {
        closesocket(client);
        return 0;
    }
    p.szNickname[NICK_LEN - 1] = 0;
    addClient(client, p.szNickname);
    printf("'%'s' dang nhap.\n", p.szNickname);

    // Server tu dong gui danh sach cho tat ca (gom ca client vua vao)
    broadcastList();

    // 2. Vong lap xu ly goi tin tu client nay
    while (recvPacket(client, &p) > 0) {
        if (p.iType == 1) {
            // Chat: szNickname = nickname DICH. Doi thanh nickname NGUOI GUI roi chuyen tiep.
            char dest[NICK_LEN];
            strncpy(dest, p.szNickname, NICK_LEN);
            dest[NICK_LEN - 1] = 0;

            SOCKET destSock = getSockByNick(dest);
            if (destSock != INVALID_SOCKET) {
                char sender[NICK_LEN];
                getNickBySock(client, sender);
                memset(p.szNickname, 0, NICK_LEN);
                strncpy(p.szNickname, sender, NICK_LEN - 1);
                sendPacket(destSock, &p);
            }
        } else if (p.iType == 2) {
            // Client xin danh sach -> gui riêng cho client nay
            Packet list;
            buildListPacket(&list);
            sendPacket(client, &list);
        }
    }

    // 3. Client thoat -> go khoi danh sach, cap nhat list cho moi nguoi
    char nick[NICK_LEN];
    getNickBySock(client, nick);
    removeClient(client);
    closesocket(client);
    printf("'%'s' dang xuat.\n", nick);
    broadcastList();
    return 0;
}

int main() {

```

```
WSADATA wsa;
WSAStartup(MAKEWORD(2, 2), &wsa);
InitializeCriticalSection(&g_cs);

SOCKET listenSock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = INADDR_ANY;
addr.sin_port = htons(SERVER_PORT);

if (bind(listenSock, (sockaddr *)&addr, sizeof(addr)) == SOCKET_ERROR) {
    printf("Loi bind: %d\n", WSAGetLastError());
    return 1;
}
listen(listenSock, SOMAXCONN);
printf("ChatServer lang nghe cong %d ...\n", SERVER_PORT);

while (1) {
    sockaddr_in cAddr;
    int len = sizeof(cAddr);
    SOCKET client = accept(listenSock, (sockaddr *)&cAddr, &len);
    if (client == INVALID_SOCKET) continue;
    HANDLE h = CreateThread(NULL, 0, ClientThread, (LPVOID)(UINT_PTR)client, 0, NULL);
    if (h) CloseHandle(h);
    else closesocket(client);
}

DeleteCriticalSection(&g_cs);
closesocket(listenSock);
WSACleanup();
return 0;
}
```

ChatClient_blocking.cpp — ChatClient — blocking (de 1)

```

// ChatClient - mo hinh BLOCKING (de chat - DE 1, Cau 1)
// Bien dich tren Windows:
//   cl ChatClient_blocking.cpp ws2_32.lib
//   g++ ChatClient_blocking.cpp -lws2_32 -o ChatClient
//
// Cung giao thuc voi ChatServer_blocking.cpp.
// Lenh nguoi dung:
//   /list           -> xin danh sach nickname dang online
//   @<nickname> <noi dung> -> gui tin nhan rieng den <nickname>
//   /quit          -> thoat
//
// Mo hinh blocking: dung 2 luong - 1 luong nhan tu server, 1 luong (main) doc ban phim.

#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <stdio.h>
#include <string.h>

#pragma comment(lib, "ws2_32.lib")

#define SERVER_PORT "8888"
#define NICK_LEN    32
#define DATA_LEN    1024

struct Packet {
    int  iType;
    int  iLength;
    char szNickname[NICK_LEN];
    char bData[DATA_LEN];
};

SOCKET g_sock;
char g_myNick[NICK_LEN];

int sendPacket(SOCKET s, Packet *p) {
    char *buf = (char *)p;
    int total = 0, n = sizeof(Packet);
    while (total < n) {
        int ret = send(s, buf + total, n - total, 0);
        if (ret <= 0) return ret;
        total += ret;
    }
    return total;
}

int recvPacket(SOCKET s, Packet *p) {
    char *buf = (char *)p;
    int total = 0, n = sizeof(Packet);
    while (total < n) {
        int ret = recv(s, buf + total, n - total, 0);
        if (ret <= 0) return ret;
        total += ret;
    }
    return total;
}

// Luong nhan goi tin tu server va hien thi
DWORD WINAPI ReceiverThread(LPVOID param) {
    Packet p;
    while (recvPacket(g_sock, &p) > 0) {
        if (p.iType == 1) {
            // szNickname = nguoi gui, bData = cau thoai
            p.szNickname[NICK_LEN - 1] = 0;
            printf("\n[%s]: %s\n> ", p.szNickname, p.bData);
        } else if (p.iType == 2) {
            int count = p.iLength / NICK_LEN;
            printf("\n--- Online (%d) ---\n", count);
            for (int i = 0; i < count; i++)

```

```

        printf(" %.s\n", NICK_LEN, p.bData + i * NICK_LEN);
        printf("> ");
    }
    fflush(stdout);
}
printf("\nMat ket noi voi server.\n");
return 0;
}

int main() {
    WSADATA wsa;
    WSStartup(MAKEWORD(2, 2), &wsa);

    char host[256];
    printf("Dia chi server: ");
    fgets(host, sizeof(host), stdin);
    host[strcspn(host, "\r\n")] = 0;

    printf("Nickname: ");
    fgets(g_myNick, sizeof(g_myNick), stdin);
    g_myNick[strcspn(g_myNick, "\r\n")] = 0;

    // Phan giai + ket noi
    addrinfo hints, *res = NULL;
    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_protocol = IPPROTO_TCP;
    if (getaddrinfo(host, SERVER_PORT, &hints, &res) != 0) {
        printf("Khong phan giai duoc dia chi server\n");
        return 1;
    }
    g_sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    if (connect(g_sock, res->ai_addr, (int)res->ai_addrlen) == SOCKET_ERROR) {
        printf("Loi ket noi: %d\n", WSAGetLastError());
        freeaddrinfo(res);
        return 1;
    }
    freeaddrinfo(res);

    // Gui goi dang nhap (type 0)
    Packet login;
    memset(&login, 0, sizeof(login));
    login.iType = 0;
    login.iLength = 0;
    strncpy(login.szNickname, g_myNick, NICK_LEN - 1);
    sendPacket(g_sock, &login);

    // Luong nhan
    HANDLE hRecv = CreateThread(NULL, 0, ReceiverThread, NULL, 0, NULL);

    printf("Da ket noi. Lenh: /list | @<nick> <noi dung> | /quit\n> ");
    fflush(stdout);

    // Vong lap doc ban phim
    char line[1200];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\r\n")] = 0;

        if (strcmp(line, "/quit") == 0) {
            break;
        }
        else if (strcmp(line, "/list") == 0) {
            Packet p;
            memset(&p, 0, sizeof(p));
            p.iType = 2;
            p.iLength = 0;
            sendPacket(g_sock, &p);
        }
        else if (line[0] == '@') {
            // @<nick> <noi dung>
            char *sp = strchr(line, ' ');
            if (!sp) { printf("Cu phap: @<nick> <noi dung>\n"); fflush(stdout); continue; }
            *sp = 0;
            char *dest = line + 1;           // bo dau '@'

```

```
    char *msg = sp + 1;

    Packet p;
    memset(&p, 0, sizeof(p));
    p.iType = 1;
    strncpy(p.szNickname, dest, NICK_LEN - 1);    // nickname DICH
    strncpy(p.bData, msg, DATA_LEN - 1);
    p.iLength = (int)strlen(p.bData);
    sendPacket(g_sock, &p);
    printf("> "); fflush(stdout);
} else if (strlen(line) > 0) {
    printf("Lenh khong hieu. Dung: /list | @<nick> <noi dung> | /quit\n> ");
    fflush(stdout);
} else {
    printf("> "); fflush(stdout);
}
}

closesocket(g_sock);
WaitForSingleObject(hRecv, 1000);
CloseHandle(hRecv);
WSACleanup();
return 0;
}
```